BE-Operator v2: An Offline Polymorphic Runtime & Evolutionary Skill Pack Architecture

Technical White Paper — Version 2.0 Draft Organization: BE-AI-Research

Shayne G. Brown - October 2026

Abstract

First-generation agentic systems treat Large Language Models (LLMs) as static tool-callers tethered to cloud APIs, rigid plugin schemas, and bloated system prompts. While commercial command-line wrappers prioritize protective guardrails for casual users, they introduce severe vendor chokepoints, context-window saturation, and brittle integration boundaries.
BE-Operator v2 (built upon the be-code-redux polymorphic runtime) introduces a paradigm shift: treating the model weights as only 50% of the equation and the execution harness as a **self-extending, local-first operating layer**. This paper outlines the architecture of BE-Operator v2, featuring **Tightly Chunked Polymorphic Skill Packs** (Markdown, JSON, TypeScript, and Go Templates), a **Two-Tier Local/Global Evolutionary Versioning System**, an **On-Device Mobile Virtual Network** (bridging native Android inference engines with Linux userland environments via local IPC), and the resulting transformation of software engineering from **static, pre-compiled applications** to **self-healing, liquid software**.

1. Introduction: Beyond the Brain in a Jar

A standalone LLM is a high-dimensional reasoning engine locked in a text-only jar. When connected to external toolkits, the model gains kinetic agency—the ability to mutate filesystems, execute network transactions, and manage infrastructure.
However, current industry harnesses suffer from three fundamental bottlenecks:

1. **Static Tool Fragility:** Traditional agents rely on hardcoded connectors or static Model Context Protocol (MCP) servers. When an external API shifts, an OS path changes, or an unmapped hardware device is encountered, the agent halts and waits for a human developer to patch the integration.
2. **Context Window Bloat (The KV-Cache Tax):** Loading dozens of static tool definitions into a system prompt consumes thousands of tokens before execution even begins. On local or mobile hardware with constrained unified memory, this degrades reasoning quality and causes severe latency spikes.
3. **Centralized Paternalism vs. Operational Leverage:** Commercial harnesses enforce opaque, cloud-side safety classifiers and constant human-in-the-loop (HITL) friction to protect novice users from brash prompting. While appropriate for consumer chat, this "blade-guard" architecture prevents unattended daemon operation, sovereign infrastructure self-healing, and deep systems engineering.

BE-Operator v2 is engineered as an industrial-grade, "governor-off" power tool for systems

architects—and as the invisible, self-healing engine beneath next-generation software applications.

2. Core Architecture: The Polymorphic Runtime

Rather than restricting an agent to pre-installed tools, BE-Operator v2 operates as a **Polymorphic Runtime**. When the operator encounters an unmapped interface, a novel binary format, or an environmental edge case, it shifts from *tool user* to *tool smith*:

\text{System Capability} = f(\text{Model Weights}) \times g(\text{Polymorphic Harness} + \text{Active Skill Chunks})

1. **Inspect & Probe:** The runtime queries the host environment, network topology, or target API.
2. **Synthesize & Hydrate:** Using polymorphic templates, the runtime generates or adapts executable code tailored to the exact host architecture (e.g., x86_64 Linux server vs. aarch64 Android Termux).
3. **Verify & Compile:** Deterministic pre-execution gates (such as TypeScript tsc type-checking and AST safety hooks) validate the synthesized tool before it touches live state.
4. **Crystallize:** Verified capabilities are written back to versioned storage, ensuring the system never pays the "reasoning tax" for the same problem twice.

3. Polymorphic Skill Packs: L1 Cache for Agent Capabilities

To eliminate cold-start synthesis overhead without flooding the model's context window, BE-Operator v2 introduces **Chunked Polymorphic Skill Packs**.
Instead of loading monolithic plugins, skill packs are broken into granular, lazy-loaded nodes. The model holds only a compact semantic index in working memory, pulling specific implementation or edge-case recovery chunks on demand.

3.1 The Four-Layer Skill Pack Stack

Each skill chunk combines four specialized formats, allowing deterministic tools and neural reasoning to share the workload cleanly:

Layer	Format	Architectural Role
Cognitive Routing	**Markdown (.md)**	Semantic hierarchy, usage heuristics, and troubleshooting trees. Parsed natively by LLMs with minimal token overhead.
Manifest & Contracts	**JSON (.json)**	Strict, machine-parseable schemas defining chunk pointers, dependency graphs, input/output contracts, and version metadata.
Zero-Token Polymorphism	**Go Templates (text/template)**	Deterministic environmental hydration ({{ .OS }}, {{ if eq .Env

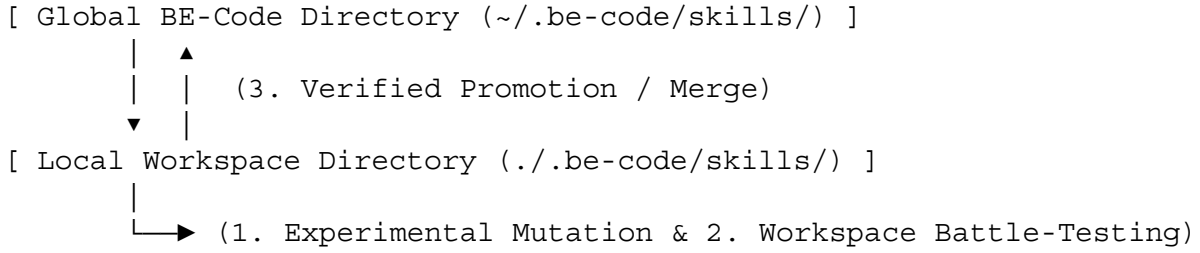
Layer	Format	Architectural Role
		"termux" }}) resolved *before* model inference or compilation, eliminating LLM boilerplate rewrites.
Typed Execution	**TypeScript (.ts)**	Strongly-typed executable payloads validated via fast compiler checks (tsc) prior to runtime registration.

3.2 Lazy-Loaded Edge-Case Resolution

During standard execution, the agent pulls only the primary execution chunk (C_0). If the runtime catches an anomalous return code or environment-specific failure (E_i), the harness queries the JSON manifest for the matching recovery chunk (C_{E_i}) and injects only those ~200–500 tokens into the active context window.

4. Two-Tier Evolutionary Versioning: Local vs. Global Scope

A self-modifying agent that overwrites its master toolset in place risks **systemic regression**—fixing a quirk in one project while breaking five others. BE-Operator v2 solves this via a **Two-Tier Lexical Versioning System** across the **Local (Project Workspace)** and **Global (BE-Code Root)** directories.



1. **Workspace Isolation (Local Tier):** When operating inside a project directory, all newly synthesized chunks, Go-template branches, and TypeScript patches are committed as versioned deltas inside the local .be-code workspace folder. Local chunks shadow global chunks without mutating the baseline installation.
2. **The Evolutionary Ratchet (Promotion Pipeline):** Once a local skill mutation passes repeated execution cycles and deterministic verification checks within a workspace, the operator (or human architect) promotes the delta into the **Global BE-Code folder**.
3. **Cross-Device Intelligence Transfer:** A heavy workstation running frontier cloud APIs or large local models can resolve complex edge cases, crystallize the solution into a global skill chunk, and sync that compact chunk to resource-constrained edge devices.

5. Edge Sovereignty: The Android Virtual Network (Termux + IPC Bridge)

A major breakthrough in BE-Operator v2 is full execution support on mobile silicon (aarch64 Android) by splitting the **Cognitive Engine** from the **Execution Harness** across Android's application sandbox.

5.1 Dual-App Topology

Running both LLM inference (llama.cpp) and a full compiler/shell harness inside a single Android process leads to severe thermal throttling, memory exhaustion, and intervention by Android's Phantom Process Killer. BE-Operator v2 establishes an **on-device virtual network** between two specialized environments:

- **The Inference Node (PocketPal / Native llama.rn Bridge):** Manages quantized GGUF weights, hardware NPU/GPU acceleration (Vulkan/OpenCL), and foreground lifecycle persistence.
- **The Operator Node (BE-Code Redux in Termux):** Provides a complete Linux userland, Node/TypeScript compilers, Go templating engine, Git versioning, and filesystem/network tools.

5.2 Local IPC & Loopback Transport

By reverse-engineering and bridging the Android IPC layer (via Android Intents, Binder ParcelFileDescriptor shared memory pipes, or local loopback/abstract Unix domain sockets), BE-Operator v2 creates an air-gapped, zero-latency inference bus directly on the device:

- **Zero Cloud Dependency:** Capable of full autonomous code synthesis and local automation in airplane mode.
- **Hybrid Routing Ready:** Keeps sensitive personal data and shell execution strictly on-device while retaining the option to route high-complexity architectural planning to frontier cloud APIs when connectivity permits.

6. Containment & Branch Pruning in "Governor-Off" Systems

Because BE-Operator v2 removes paternalistic cloud refusals to maximize operational leverage, security shifts from *prompt politeness* to **deterministic infrastructure containment** ("pruning anomalous branches"):

- **Read-Only Core Kernel:** While the agent has read/write privileges over local workspace skill chunks, the core BE-Operator orchestration kernel and version-control hooks are mounted read-only (ro).
- **Deterministic Pre-Execution Gates:** Synthesized TypeScript chunks must pass static AST inspection and tsc compilation before execution. Destructive shell patterns or unauthorized network egress triggers an immediate circuit breaker.
- **Instant Version Rollback:** Because every skill mutation and workspace state transition is version-tracked locally, anomalous agent drift can be pruned instantaneously by reverting to the prior verified checkpoint.

7. The Post-Application Paradigm: What This Means

for Future Software Development

For the past fifty years, software development has followed a rigid, static lifecycle: **Design \rightarrow Hardcode \rightarrow Compile \rightarrow Ship \rightarrow Break \rightarrow Patch**. Developers spend 80% of their engineering hours writing boilerplate "glue code"—REST adapters, data parsers, state migrations, and UI forms—anticipating every possible edge case before the user ever launches the binary.
BE-Operator v2 fundamentally dismantles this model. When the underlying runtime is polymorphic and backed by self-updating skill packs, software transitions from **Rigid Pre-Compiled Binaries** to **Liquid, Self-Adapting Systems**.

7.1 From Static Binaries to "Thin Intent Shells"

In the BE-Operator v2 paradigm, an "application" is no longer a 300MB bundle of hardcoded business logic. Instead, an application consists of three lightweight layers:

1. **A Thin Declarative UI / Intent Layer:** A clean visual interface that captures user goals and displays structured state.
2. **A Curated Baseline of Polymorphic Skill Packs:** The domain-specific starting knowledge (e.g., financial parsing, network management, media transcoding).
3. **The BE-Operator Runtime:** The local engine that dynamically bridges the gap between the user's intent and the messy reality of their local environment, OS, and external data sources.

Traditional Software Engineering	BE-Operator v2 Development Paradigm
Hardcoded Integrations: Every API endpoint, file format, and device protocol must be anticipated and coded at build time.	**Just-in-Time (JIT) Synthesis:** Unmapped APIs, new file schemas, and unknown LAN devices are probed and integrated at runtime.
Brittle Failure Modes: An upstream schema change throws an unhandled exception; the app stays broken until the next patch cycle.	**Autonomous Self-Healing:** An exception triggers the diagnostic skill chunk; the runtime patches the TypeScript adapter locally and resumes execution.
Monolithic Bloat: Apps bundle thousands of unused libraries to cover hypothetical user edge cases.	**Chunked On-Demand Footprint:** Only the active 5KB skill chunk resides in memory; edge-case code is synthesized or lazy-loaded only when encountered.
Centralized Cloud Lock-In: User data is siphoned to vendor servers to run proprietary backend logic.	**Local-First Sovereignty:** Compute, inference (via mobile/local IPC), and mutated skill state remain entirely on the user's hardware.

7.2 The Death of the "Integration Backlog"

In traditional SaaS and consumer software, engineering teams drown in maintenance backlogs—updating OAuth flows, fixing broken web scrapers, supporting new smart-home firmware, or adding regional bank CSV formats.
Under BE-Operator v2, **maintenance becomes an autonomous background process of the runtime itself**. When an external service mutates its response payload, the local BE-Operator instance catches the contract violation via its JSON schema check, loads its recovery chunk, inspects the new payload structure, rewrites the TypeScript parser in the local .be-code workspace, verifies it via tsc, and completes the user's task. The user never sees an error

screen, and the developer never gets paged at 2:00 AM.

7.3 The Inversion of the Developer's Role: From Bricklayer to Evolutionary Architect

This paradigm does not eliminate software engineers; it elevates them from writing repetitive glue code to designing **evolutionary boundaries and skill topologies**:

- **Authoring Seed DNA (Skill Packs):** Instead of writing 50,000 lines of static integration code, developers craft high-leverage, chunked Skill Packs—defining clean Markdown routing trees, strict JSON contracts, and cross-platform Go templates that give the runtime an optimal starting point.
- **Designing Verification Harnesses:** Because the runtime writes its own implementation details, the developer's primary job shifts to writing **deterministic fitness functions**—the AST rules, type interfaces, and sandbox boundaries that define what a "valid, safe mutation" looks like before a local skill is allowed to execute or promote to the global tier.
- **Curating the Evolutionary Ratchet:** Developers review high-performing local workspace mutations generated across edge deployments and merge the cleanest solutions into the master global skill registry.

7.4 Inversion of Software Economics (Zero Marginal Cost of Adaptation)

Historically, bespoke software—software tailored to an individual's exact hardware, niche workflow, or proprietary data formats—was prohibitively expensive, reserved only for enterprise budgets. Consumers were forced to bend their habits to fit mass-market apps.
Because BE-Operator v2 can run locally on commodity hardware (down to an Android phone via the Termux + PocketPal IPC bridge) and crystallize its work into reusable chunks, **the marginal cost of custom software adaptation drops to near zero**. Every consumer device effectively has a senior systems engineer living inside its runtime, continuously tailoring the software to the exact contours of its owner's digital life.

8. Next-Generation Consumer Applications

When the BE-Operator v2 paradigm is applied to everyday consumer categories, it enables classes of software that were previously impossible to build or maintain:

8.1 The Universal Local Smart Home & IoT Translator

Instead of failing when a user buys an unsupported sensor, LAN appliance, or 3D printer, a BE-Operator home hub scans the local subnet, inspects the device's HTTP/MQTT/mDNS broadcasts, synthesizes a local adapter chunk, and integrates the device natively—operating 100% locally without cloud dependencies.

8.2 Self-Healing Personal Automations (The Anti-Zapier)

Consumer automation tools break whenever a website layout, receipt format, or API endpoint changes. A personal BE-Operator running on-device detects the parsing failure, loads its

diagnostic skill chunk, rewrites the extractor to match the new schema, versions the fix locally, and continues running seamlessly.

8.3 Sovereign Personal Finance & Administrative Triage

By running entirely on a user's smartphone via the Termux + Local Inference bridge, BE-Operator v2 can ingest bank PDFs, brokerage statements, and private emails using locally synthesized parser chunks—delivering autonomous financial tracking and inbox triage with zero third-party cloud data exposure.

8.4 Liquid "Bespoke Micro-Apps"

Rather than subscribing to ten rigid SaaS apps, users can describe a hyper-specific workflow (e.g., cross-referencing local ferry telemetry, calendar events, and home charging schedules). BE-Operator v2 assembles existing global skill chunks, synthesizes the missing glue logic in a local workspace, and presents a working, self-maintaining micro-application in seconds.

8.5 The Decentralized Skill Pack Marketplace ("DNA Distribution")

Rather than distributing 200MB monolithic apps, BE-Operator v2 enables a **Community Skill Registry**. When one user's operator solves a novel API integration or device protocol and promotes the verified chunk, that 10KB polymorphic skill pack can be shared globally—allowing millions of small, local mobile models to inherit complex capabilities instantly without burning local battery or compute on discovery.

9. Roadmap & Conclusion

BE-Operator v2 proves that the future of autonomous AI and software engineering is not larger, locked-down cloud monoliths or static app stores, but **agile, self-evolving harnesses** paired with compact, composable skill packs. By combining polymorphic code synthesis, two-tier lexical versioning, and on-device mobile IPC bridging, BE-Operator v2 lays the foundation for the post-application era—delivering sovereign, self-healing software to both systems engineers and everyday consumers.